UNOESTE - UNIVERSIDADE DO OESTE PAULISTA
CIÊNCIA DA COMPUTAÇÃO

EVANDRO TAROCO DE CARVALHO
LUIZ GUSTAVO RENNER SCATALON
MATHEUS CHIZZOLINI MONTEIRO

RELATÓRIO PROJETO BIMESTRAL - SOLUÇÃO COM REDES NEURAIS MLP PARA A BASE DE DADOS: RAIN IN AUSTRALIA

Presidente Prudente - SP
2026

1- INTRODUÇÃO

A previsão de chuva é um problema clássico de aprendizado de máquina que possui grande relevância prática, impactando diretamente setores como agricultura, logística, planejamento urbano e gestão de recursos hídricos. A capacidade de antecipar se haverá precipitação no dia seguinte permite que indivíduos e organizações tomem decisões mais informadas e preventivas.

Este trabalho desenvolve uma solução baseada em Redes Neurais do tipo Multilayer Perceptron (MLP) para prever a ocorrência de chuva no dia seguinte, utilizando dados meteorológicos históricos da Austrália. O problema é tratado como uma tarefa de classificação binária, em que o modelo deve determinar, a partir de variáveis climáticas coletadas ao longo de um dia, se haverá ou não chuva no dia seguinte.

A escolha da base de dados Rain in Australia se justifica por diversas razões. Em primeiro lugar, trata-se de um conjunto de dados público e bem documentado, disponível na plataforma Kaggle, com registros reais coletados pelo Bureau of Meteorology da Austrália. Em segundo lugar, o volume de dados é expressivo (cerca de 145 mil observações), o que favorece o treinamento de redes neurais. Por fim, a variedade de variáveis disponíveis, que incluem temperatura, umidade, velocidade do vento, pressão atmosférica e nebulosidade, confere ao problema uma complexidade adequada para explorar as capacidades de uma MLP.

O objetivo principal é construir um modelo capaz de aprender padrões climáticos relevantes e generalizar bem para novos dados, sendo posteriormente integrado a uma interface interativa que permita ao usuário realizar previsões a partir de entradas personalizadas.

2- BASE DE DADOS

2.1- ORIGEM E DESCRIÇÃO

A base de dados utilizada é o conjunto Rain in Australia, disponível publicamente no Kaggle. Os dados foram originalmente coletados pelo Bureau of Meteorology da Austrália e abrangem registros meteorológicos diários de diversas cidades australianas ao longo de aproximadamente dez anos. O dataset contém cerca de

145 mil observações e 23 colunas, das quais 21 são variáveis preditoras e 1 é a variável-alvo (RainTomorrow).

As variáveis disponíveis incluem:

- Temperatura mínima e máxima do dia (MinTemp, MaxTemp);
- Volume de chuva registrado no dia (Rainfall);
- Evaporação, horas de sol e velocidade do vento (Evaporation, Sunshine, WindGustSpeed);
- Direção do vento medida às 9h e às 15h (WindDir9am, WindDir3pm);
- Umidade relativa do ar às 9h e às 15h (Humidity9am, Humidity3pm);
- Pressão atmosférica às 9h e às 15h (Pressure9am, Pressure3pm);
- Cobertura de nuvens às 9h e às 15h (Cloud9am, Cloud3pm);
- Temperatura às 9h e às 15h (Temp9am, Temp3pm);
- Indicador binário de chuva no dia atual (RainToday);
- Localidade da estação meteorológica (Location).

A variável-alvo RainTomorrow é binária, assumindo os valores "Yes" (choverá amanhã) ou "No" (não choverá amanhã). A análise exploratória revelou um desequilíbrio significativo entre as classes: aproximadamente 78% das observações correspondem à classe "Não Chove" e apenas 22% à classe "Chove". Esse desbalanceamento é um fator determinante nas decisões de modelagem, especialmente na escolha de métricas de avaliação e nas técnicas de otimização do modelo

2.2- PRÉ-PROCESSAMENTO

O pré-processamento dos dados foi realizado em etapas sequenciais, garantindo a integridade e a adequação dos dados para o treinamento da rede neural.

As etapas realizadas foram as seguintes:

- Remoção da coluna Date: a data foi descartada por não conter informação diretamente utilizável pelo modelo no formato original.
- Remoção de registros com alvo ausente: linhas em que a variável RainTomorrow era nula foram removidas, pois não contribuem para o aprendizado supervisionado.

- Imputação de valores ausentes em variáveis numéricas: os valores faltantes foram substituídos pela mediana de cada coluna, estratégia robusta à presença de outliers.
- Imputação de valores ausentes em variáveis categóricas: os valores faltantes foram substituídos pela moda de cada coluna.
- Codificação de variáveis binárias: as colunas RainToday e RainTomorrow, que assumiam os valores "Yes" e "No", foram convertidas para valores numéricos 1 e 0 respectivamente, utilizando LabelEncoder.
- One-Hot Encoding nas variáveis categóricas nominais: as colunas Location, WindGustDir, WindDir9am e WindDir3pm foram transformadas em variáveis dummy por meio do método get_dummies, com a opção drop_first=True para evitar multicolinearidade.
- Divisão treino/teste: os dados foram divididos na proporção 80/20, resultando em 113.754 amostras de treinamento e 28.439 amostras de teste.
- Normalização com StandardScaler: as features foram padronizadas para média zero e desvio padrão unitário. O scaler foi ajustado exclusivamente sobre os dados de treinamento e aplicado tanto ao conjunto de treino quanto ao de teste, prevenindo vazamento de informação (data leakage).

Após o pré-processamento, o conjunto de dados passou a contar com 110 features de entrada, resultado da expansão produzida pelo One-Hot Encoding das variáveis categóricas.

	feature	type	treatment
0	MinTemp	numerica	mediana + StandardScaler
1	MaxTemp	numerica	mediana + StandardScaler
2	Rainfall	numerica	mediana + StandardScaler
18	WindGustDir	categorica	moda + LabelEncoder/get_dummies
19	WindDir9am	categorica	moda + LabelEncoder/get_dummies
20	WindDir3pm	categorica	moda + LabelEncoder/get_dummies

3- MODELO MLP

3.1- ARQUITETURA

A rede neural desenvolvida é um Multilayer Perceptron (MLP) feedforward totalmente conectado, implementado com a biblioteca Keras (TensorFlow). A arquitetura foi projetada para equilibrar capacidade de aprendizado e generalização, sendo composta por três camadas ocultas com número decrescente de neurônios:

- Camada de entrada: 110 neurônios, correspondendo ao número de features após o pré-processamento;
- Primeira camada oculta: 128 neurônios, seguida de Dropout com taxa de 30%;
- Segunda camada oculta: 64 neurônios, seguida de Dropout com taxa de 20%;
- Terceira camada oculta: 32 neurônios, seguida de Dropout com taxa de 20%;
- Camada de saída: 1 neurônio, responsável por produzir a probabilidade de chuva no dia seguinte.

O modelo totaliza 24.577 parâmetros treináveis, um número relativamente compacto que favorece a eficiência computacional sem comprometer a capacidade de aprendizado para o problema em questão.

3.2- FUNÇÕES DE ATIVAÇÃO

As camadas ocultas utilizam a função de ativação ReLU (Rectified Linear Unit), definida como $f(x) = \max(0, x)$. A ReLU é amplamente adotada em redes neurais profundas por sua eficiência computacional, por mitigar o problema do gradiente desvanecente e por apresentar bom desempenho empírico em uma grande variedade de tarefas de classificação.

A camada de saída utiliza a função Sigmoid, que mapeia qualquer valor real para o intervalo (0, 1), interpretado como a probabilidade da classe positiva (Chove = 1). A classificação final é obtida aplicando um limiar de 0,5: valores acima são classificados como "Chove" e valores abaixo como "Não Chove".

3.3- TÉCNICAS DE OTIMIZAÇÃO E HIPERPARÂMETROS

O treinamento foi configurado com os seguintes hiperparâmetros e técnicas:

- Otimizador Adam: algoritmo de otimização adaptativa que combina os benefícios do RMSProp e do Momentum, ajustando a taxa de aprendizado individualmente para cada parâmetro. É a escolha padrão para a maioria dos problemas de classificação com redes neurais.
- Função de perda Binary Cross-Entropy: adequada para problemas de classificação binária, mede a divergência entre a distribuição prevista e a distribuição real das classes.
- Dropout: técnica de regularização que desativa aleatoriamente uma fração dos neurônios durante o treinamento, reduzindo o overfitting ao forçar a rede a aprender representações mais robustas. Taxas de 30% na primeira camada e 20% nas demais foram aplicadas.
- Pesos de classe (class_weight): devido ao desequilíbrio entre as classes, foram calculados pesos inversamente proporcionais à frequência de cada classe utilizando a função compute_class_weight da biblioteca scikit-learn. A classe "Chove" recebeu peso aproximado de 2,23 e a classe "Não Chove" peso de 0,64, penalizando mais os erros cometidos na classe minoritária.
- Número de épocas: 30 épocas de treinamento, com monitoramento da acurácia de validação ao longo do processo.
- Tamanho do batch: 128 amostras por atualização de gradiente, escolha que equilibra estabilidade de convergência e tempo de treinamento.

4- AVALIAÇÃO DO MODELO

4.1- VALIDAÇÃO CRUZADA COM STRATIFIEDKFOLD

Para estimar o desempenho real do modelo e garantir que os resultados não dependessem de uma divisão específica dos dados, foi utilizada a validação cruzada estratificada com 3 folds (StratifiedKFold). A estratificação garante que a proporção original das classes seja mantida em cada fold, aspecto especialmente relevante dado o desequilíbrio da base de dados.

Os resultados da validação cruzada foram os seguintes:

- Fold 1 — melhor acurácia de validação: 81,74%;
- Fold 2 — melhor acurácia de validação: 81,91%;
- Fold 3 — melhor acurácia de validação: 81,97%;
- Acurácia média de validação: 81,87% ± 0,10%.

O desvio padrão muito baixo entre os folds (0,10%) indica que o modelo é estável e generaliza de forma consistente, independentemente da partição dos dados utilizada. Esse comportamento é um indicativo de que o modelo não está sofrendo overfitting severo.

4.2- DESEMPENHO NO CONJUNTO DE TESTE

Após a validação cruzada, um modelo final foi treinado com todos os dados de treino disponíveis e avaliado no conjunto de teste separado (28.439 amostras, nunca vistas durante o treinamento). Os resultados obtidos foram:

- Acurácia: 80,87%;
- Precisão: 54,83%;
- Recall: 80,51%;
- F1-Score: 65,24%.

Para interpretar corretamente esses números, é fundamental considerar o contexto do problema. Em previsão de chuva, o erro mais custoso é o Falso Negativo, ou seja, prever que não vai chover quando na verdade vai. Esse tipo de erro pode levar o usuário a não se preparar para a chuva, com consequências práticas mais severas do que prever chuva quando o dia estará seco. Por esse motivo, o Recall é a métrica mais relevante neste cenário.

O Recall de 80,51% significa que o modelo identificou corretamente 4 em cada 5 dias que realmente teriam chuva. Esse resultado é considerado positivo para o domínio do problema. A Precisão mais baixa (54,83%) indica que, dos dias em que o modelo previu chuva, pouco mais da metade de fato choveu, ou seja, há uma quantidade relevante de Falsos Positivos. Essa é uma consequência direta e esperada do uso de pesos de classe: ao penalizar mais os erros na classe

minoritária, o modelo se torna mais sensível à chuva, aumentando o Recall à custa de uma redução na precisão.

O F1-Score de 65,24% representa a média entre Precisão e Recall, sendo uma métrica equilibrada para cenários com classes desbalanceadas. O valor obtido está em linha com o esperado dado o grau de desbalanceamento da base.

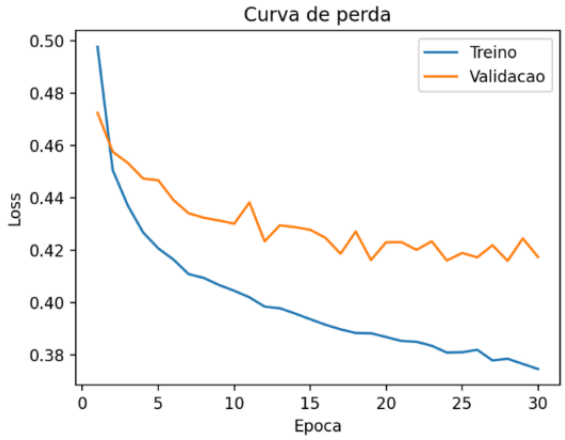
	precision	recall	f1-score	support
0	0.9359	0.8016	0.8636	22098
1	0.5391	0.8087	0.6469	6341
accuracy	0.8032	0.8032	0.8032	0.8032
macro avg	0.7375	0.8052	0.7553	28439
weighted avg	0.8474	0.8032	0.8153	28439

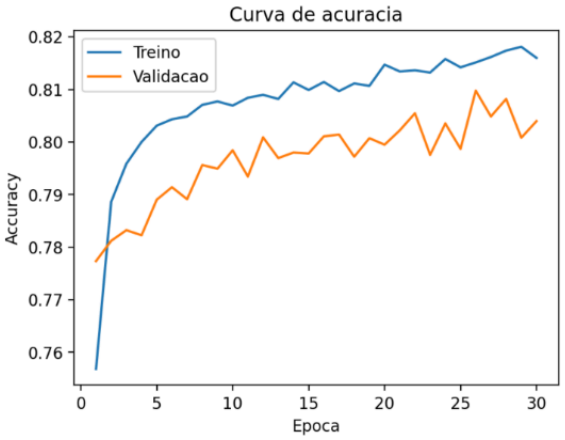
Acuracia	Precisao	Recall	F1-score
0.8032	0.5391	0.8087	0.6469

4.3- CURVAS DE APRENDIZADO

As curvas de perda (Loss) e de acurácia ao longo das épocas de treinamento foram registradas tanto para os dados de treino quanto para os de validação. Em ambas as curvas, observa-se que o modelo converge progressivamente durante as 30 épocas, sem sinais evidentes de overfitting: a perda de validação acompanha a tendência decrescente da perda de treinamento e a acurácia de validação cresce em paralelo com a acurácia de treino, sem divergir de forma significativa.

Esse comportamento confirma que o Dropout e o uso de pesos de classe contribuíram para uma regularização adequada do modelo, mantendo a capacidade de generalização sem memorizar os dados de treinamento.

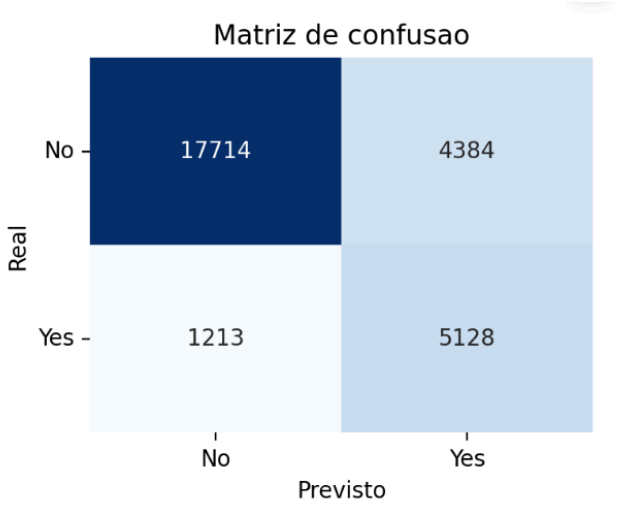




4.4- MATRIZ DE CONFUSÃO

A matriz de confusão permite visualizar a distribuição dos acertos e erros do modelo separadamente para cada classe. A análise da matriz confirmou os números apresentados nas métricas: o modelo acerta com alta taxa os dias sem chuva (classe majoritária) e demonstra boa sensibilidade aos dias com chuva (classe minoritária), com um volume de Falsos Positivos que é a principal fonte de redução da Precisão.

O perfil de erros é consistente com as prioridades do problema: o modelo tende a "errar pelo lado da cautela", prevendo chuva com mais frequência do que realmente ocorre, o que é preferível a ignorar dias que teriam precipitação.



5- DESENVOLVIMENTO DA APLICAÇÃO

O desenvolvimento da aplicação foi realizado com a biblioteca Streamlit, uma ferramenta open source do ecossistema Python voltada para a criação de interfaces web de forma simples e rápida. A escolha dessa tecnologia foi adequada ao projeto por permitir a construção de uma aplicação funcional sem a necessidade de conhecimentos em front-end, como HTML, CSS ou JavaScript. Além disso, por utilizar a mesma linguagem do restante do sistema, o Streamlit facilitou a integração entre a interface, o tratamento dos dados e o modelo de rede neural MLP.

A aplicação foi estruturada para separar o processo de treinamento da etapa de uso da interface. O treinamento do modelo é executado por um script específico no terminal, responsável por carregar a base de dados, aplicar o pré-processamento, treinar a rede MLP e salvar os artefatos necessários, como o modelo final, o scaler e os arquivos com métricas e histórico. Essa separação foi adotada para tornar a interface mais leve, rápida e adequada ao objetivo do trabalho, que é permitir testes do modelo já treinado.

Na interface, o usuário pode visualizar informações importantes sobre o projeto, como dados gerais da base utilizada, quantidade de valores nulos, métricas de desempenho, matriz de confusão e curvas de aprendizado do modelo. Dessa forma, a aplicação não apenas executa previsões, mas também ajuda a apresentar de forma clara os resultados obtidos durante o desenvolvimento, servindo como apoio na demonstração final do projeto.

O funcionamento principal da interface está no formulário de entrada de dados, em que o usuário preenche os atributos meteorológicos necessários para realizar uma previsão. A partir desses valores, a aplicação aplica o mesmo fluxo de transformação utilizado no treinamento e então envia os dados para o modelo MLP, que retorna a previsão para a variável RainTomorrow. O resultado é exibido de forma objetiva, mostrando tanto a classe prevista quanto a probabilidade associada.

Para facilitar os testes, a interface também conta com um botão de preenchimento automático com valores aleatórios baseados em registros reais da base. Esse recurso torna a demonstração mais prática, pois evita o preenchimento manual de

todos os campos em cada teste. Assim, a aplicação cumpre o objetivo proposto no projeto, oferecendo uma interface simples, funcional e integrada ao modelo treinado.

6- CONCLUSÃO

Este trabalho apresentou o desenvolvimento de uma solução baseada em Redes Neurais MLP para a previsão de chuva no dia seguinte, utilizando dados meteorológicos históricos da Austrália. O pipeline completo foi implementado em Python, abrangendo desde o pré-processamento dos dados até a disponibilização do modelo em uma interface interativa.

Os resultados obtidos são positivos e adequados ao contexto do problema. O modelo final apresentou Recall de 80,51% no conjunto de teste, identificando corretamente a grande maioria dos dias com chuva, que é a métrica prioritária para esse tipo de aplicação. A validação cruzada com StratifiedKFold confirmou a estabilidade do modelo, com acurácia média de 81,87% e desvio padrão inferior a 0,1% entre os folds.

O desequilíbrio entre as classes foi tratado com a aplicação de pesos de classe, o que elevou o Recall à custa de uma redução esperada na precisão. Essa escolha reflete uma decisão consciente sobre o perfil de erros desejável no contexto da previsão meteorológica, onde o custo de não alertar sobre chuva é maior do que o custo de um alerta falso.

Como possíveis melhorias futuras, destacam-se:

- Ampliação do número de épocas de treinamento e exploração sistemática de hiperparâmetros, como taxa de aprendizado e arquitetura da rede, com técnicas de busca como Grid Search ou Bayesian Optimization;
- Engenharia de features adicionais, como a codificação cíclica de variáveis temporais (mês, estação do ano) ou a criação de médias móveis para capturar tendências ao longo dos dias;
- Ajuste do limiar de classificação: em vez de fixar o limiar em 0,5, seria possível otimizá-lo com base na curva ROC para maximizar o Recall com controle sobre a Precisão;

- Comparação com outros modelos, como Gradient Boosting e Random Forest, para verificar se uma abordagem baseada em árvores oferece ganhos significativos no problema específico.

Em síntese, o projeto atingiu seus objetivos, entregando um modelo funcional, avaliado com rigor estatístico e integrado a uma interface acessível ao usuário final.